

Learning Rate Sensitivity in Adaptive Optimizers

Y malin Awhansou, Marceau Bouilly, Charline Huangfu
School of Computer and Communication Sciences, EPFL, Switzerland
CS 439 - Optimization for Machine Learning

Abstract—Adaptive optimization algorithms are widely used in deep learning for their fast convergence and robustness. Despite this adaptivity, the choice of the *initial* learning rate remains a key hyperparameter that significantly affects convergence behavior, training time and generalization performance. In this work, we study the effect of varying the initial learning rate on these optimizers when training a CNN on the CIFAR-10 dataset. Using a consistent experimental setup, we compare performance across accuracy and training time, highlighting practical trade-offs and providing tuning recommendations. Our code is available here.

I. INTRODUCTION

Optimizers are a fundamental component in training machine learning models. Adaptive methods such as Adam [1] have become standard choices due to their ability to adjust learning rates dynamically for each parameter, often resulting in faster convergence and reduced manual tuning compared to traditional methods like stochastic gradient descent (SGD).

Yet, despite their adaptivity, these optimizers remain highly sensitive to one key hyperparameter: the initial learning rate. If set too high, training can diverge, if too low, it may converge slowly. This makes learning rate selection a persistent challenge. Indeed as Wilson et al. [2] highlight, adaptive optimizers can still require careful tuning of the learning rate selection and, in some cases, may even generalize worse than SGD.

Problem. How does the initial learning rate influence training stability, convergence speed and generalization in adaptive optimizers?

To address this, our contributions are threefold. First, we systematically compare the sensitivity of eight popular optimizers—AdaGrad [3], RMSProp [4], Adam [1], AdamW [5], AMSGrad [6], RAdam [7], NAdam [8] and SGD [9] (as a baseline)—to variations in the initial learning rate. Second, we analyze the trade-off between learning rate and training time. Third, we extend this analysis by introducing additional hyperparameters to explore interactions beyond learning rate, offering more comprehensive tuning guidance for practitioners.

The remainder of this paper is structured as follows: Section II describes the experimental design, Section III presents our empirical results, Section IV discusses the implications of our findings and Section V concludes.

II. MODELS AND METHODS

A. Model and Dataset

We use a standard Convolutional Neural Network (CNN) for image classification on the CIFAR-10 dataset, which contains 60,000 color images of size 32×32 across 10 classes.

The CNN consists of two convolutional layers with 64 and 128 filters of size 3×3, respectively and input images are normalized using mean and standard deviation of 0.5 for each RGB channel.

To ensure consistency, the architecture is fixed across all experiments to isolate the effects of different optimization methods.

All optimizers are implemented using the PyTorch library.

B. Role of Initial Learning Rate in Adaptive Methods

In Adam [1], parameter updates follow the rule:

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second moments of the gradients, and η is the initial learning rate.

Despite its adaptivity, α acts as a global scaling factor. A too-large α can override the benefits of the moment estimates, leading to unstable updates, too small, and the updates become excessively slow. Thus, this highlights why α remains a critical hyperparameter even in adaptive settings.

C. Experimental Protocol — Fixed Grid Search

In our first set of experiments, we analyze the effect of the initial learning rate across different optimizers. All models are trained for 20 epochs with a batch size of 256. The random seed is fixed for reproducibility and experiments are repeated with 3 different seeds. Final results are averaged over these runs.

We sweep over five learning rates sampled logarithmically between 10^{-5} and 10^{-1} :

$$\text{LR grid} = \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$$

This range includes the default initial learning rates proposed in the original publications. For instance, as stated in the Adam paper, “Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ ” [1]. These values are widely adopted as standard defaults in most deep learning libraries, including PyTorch [10]. Similarly, other optimizers also come with standard default learning rates around 0.001, as summarized in [11].

All other hyperparameters are kept at their default values from PyTorch, which follow the original recommendations.

D. Experimental Protocol — Hyperparameter Optimization with Optuna

In a second stage, we go beyond learning rate and explore interactions between multiple optimizer hyperparameters. For this, we employ *Optuna*.

For each optimizer, Optuna searches to maximize test accuracy over a defined space that includes:

- Initial learning rate (10^{-5} to 10^{-1} , log scale)
- Weight decay (10^{-5} to 10^{-1} , log scale)
- Momentum parameters (e.g., β_1 , β_2 for Adam variants)
- α (for RMSProp)

Each trial is run for 10 epochs, with the fixed seed. Optuna evaluates test accuracy over epochs, helping identify optimal configurations and illustrating trade-offs. For each optimizer, we used Optuna to test at most 20 configurations, pruning from the start those that are not optimal to be more efficient in the search.

III. RESULTS

A. Analysis of the First Experiment Results

1) *Comparison of Optimizers: Loss According to Learning Rate* : As shown in Figure 1, for most optimizers tested - Adam, RMSprop, AdamW, Nadam, RAdam and AMSGrad- the best performance was observed at a learning rate of 10^{-3} , which fortunately corresponds to their commonly used default setting. Beyond this point, performance generally degrades as the learning rate increases or decreases, highlighting the sensitivity of these methods to proper tuning.

Interestingly, RAdam showed slightly more stable behavior across learning rates, with a slower degradation compared to others, suggesting some robustness in its rectified update mechanism.

Two optimizers stand out in this comparison : Adagrad, which underperforms relative to others, confirming its tendency to suffer from aggressive learning rate decay over time, and SGD, which shows strong performance at a higher learning rate of 0.1, outperforming adaptive methods at their optimal settings. This supports the argument by [2] that adaptive optimizers do not always generalize better.

Actually, RAdam was designed to deactivate the adaptive learning rate scaling when its variance is unreliable during early training. Indeed, RAdam introduces a rectification term r_t compared to Adam, adjusting the update rule as follows:

$$\theta_{t+1} = \theta_t - \eta \cdot r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}},$$

where r_t controls the amount of adaptive scaling based on the variance estimate. When the variance estimate is unstable at early steps, r_t reduces the adaptive effect, causing the update to behave more like SGD with momentum. This prevents unstable large updates at the beginning of training [7]. This mechanism explains why RAdam shows more stable performance across a range of learning rates.

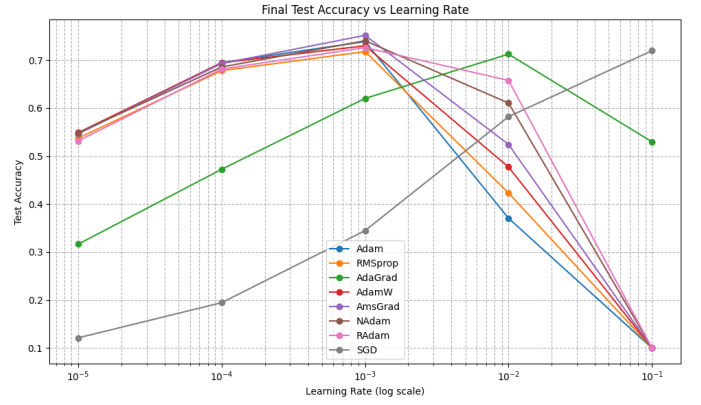


Fig. 1. Loss According to Learning Rate for 8 optimizers

On the other hands, Adagrad adapts the learning rate for each parameter by accumulating the squared gradients over time:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t, \quad \text{where } G_t = \sum_{i=1}^t g_i^2.$$

As G_t accumulates without decay, the effective learning rate continuously decreases, which can lead to overly conservative updates and poor convergence. Therefore, the initial learning rate is not enough to prevent this progressive decay and tuning the learning rate alone cannot fix this fundamental problem with AdaGrad's update rule. This behavior is discussed and further analyzed in [12].

Overall, these results underline the importance of tuning even widely-used optimizers. Fortunately, the default settings are generally the most effective.

2) *Focus on Loss Function at Learning Rate 10^{-2}* : Let's focus on the loss function at 10^{-2} (Figure 8), which reveals significant differences in optimizer behavior.

At this learning rate, RAdam demonstrates the best performance among all tested optimizers in terms of loss, highlighting its robustness and stability even at relatively high learning rates. This aligns with its variance rectification mechanism, which helps prevent unstable updates during early training [7]. This is particularly noteworthy given that some other optimizers, such as RMSprop, exhibit a peak in loss during the initial epochs, indicating early instability likely caused by its sensitivity to noisy gradient variance estimates before the moving averages stabilize.

Indeed, mathematically, RMSprop [4] computes an exponentially weighted moving average of squared gradients:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2,$$

with $v_0 = 0$ causing a biased, noisy variance estimate early in training. This makes the effective learning rate,

$$\hat{\eta}_t = \frac{\eta}{\sqrt{v_t + \epsilon}},$$

fluctuate strongly, especially for higher initial η like 10^{-2} , leading to unstable updates and a loss spike 8.

For a broader perspective, including loss and accuracy trends across all learning rates, please refer to the appendix.

3) Trade off between learning rate and training time:

We compared how quickly each optimizer reaches 65% test accuracy depending on the learning rate.

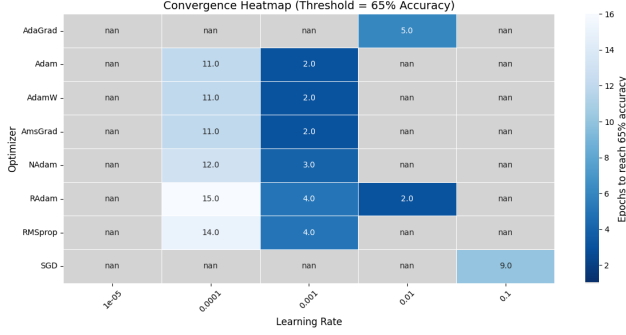


Fig. 2. Convergence heatmap showing epochs needed to reach 65% accuracy for various optimizers across learning rates.

Figure 2 illustrates the trade-off between learning rate selection and convergence speed for the eight optimizers. Adaptive optimizers converge fastest at moderate learning rates ($\sim 10^{-4}$ to 10^{-3}), but RAdam shows slightly slower convergence at 10^{-4} , reflecting its variance rectification mechanism. Notably, SGD only converges at a high learning rate (0.1), supporting its ability to achieve better generalization [2].

B. Analysis of the Second Experiment Results

For the results of this part, we will focus on two very different algorithms: AdaGrad, considered the first adaptive optimizer and RAdam, one of the most recent.

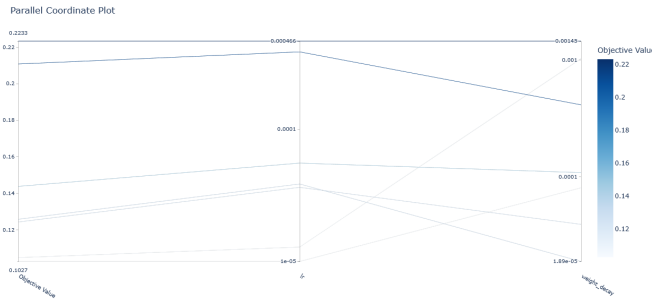


Fig. 3. Influence of AdaGrad hyperparameters on the accuracy test over 100 (objective value)

As we can see by comparing Figures 3 and 4, the difference between the two optimizers is significant. First of all, the accuracy of RAdam is overall much better than that of AdaGrad regardless of the hyperparameters. In 10 epochs, it manages to achieve accuracy of 73% while AdaGrad struggles to reach

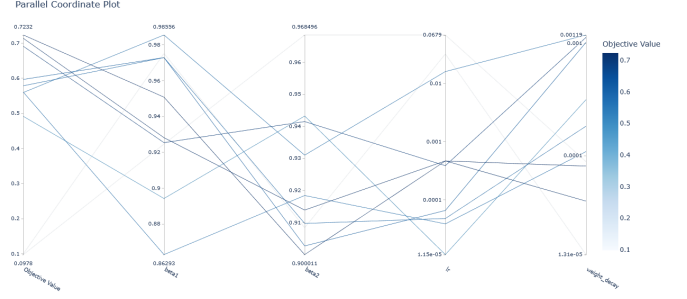


Fig. 4. Influence of RAdam hyperparameters on the accuracy test over 100 (objective value)

22%. Furthermore, even if its accuracy decreases significantly for high learning rates (greater than 10^{-2} , it is much more robust than AdaGrad whose learning rate has a huge influence on its performance and which is not optimal below 10^{-4} . This shows the extent of progress made on adaptive gradient algorithms which have continued to evolve since the appearance of AdaGrad. As shown in the hyperparameter importance graphs, new architectures like RAdam figures 15 have made it possible to reduce the importance of the initial learning rate compared to AdaGrad figures 16.

IV. DISCUSSION

While our study investigate the impact of the initial learning rate on model performance and robustness on the CIFAR-10 classification task, its scope is limited to a single dataset and architecture. As emphasized by Schmidt et al. [13], optimizer performance is highly problem-dependent. Figure 4 of their paper even shows that our task, most similar to their fourth problem, seems to be among the least discriminatory tasks.

This suggests that our setup may favor conclusions that are specific to well-behaved tasks. As a consequence, while our findings may provide practical insight for similar classification tasks, they should not be overgeneralized to other domains where optimizer behavior can diverge more markedly.

V. SUMMARY

This work evaluated the performance of eight popular optimizers across a range of initial learning rates on the CIFAR-10 dataset using a CNN architecture. Our result confirm that the commonly used default learning rate of 10^{-3} generally yields the best performance in terms of accuracy and convergence speed. RAdam stood out for its robustness, likely due to its variance rectification. Conversely, Adagrad consistently underperformed due to its aggressive learning rate decay, while SGD performed well at higher initial learning rate (0.1), supporting findings that adaptive optimizers do not always generalize better [2]. These insights highlight the critical importance of optimizer and learning rate selection for effective training and provide practical guidance for tuning deep learning models.

REFERENCES

- [1] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *International Conference on Learning Representations (ICLR)*, 2015, <https://arxiv.org/abs/1412.6980>.
- [2] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, "The marginal value of adaptive gradient methods in machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, <https://arxiv.org/abs/1705.08292>.
- [3] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011. [Online]. Available: <https://jmlr.org/papers/volume12/duchi11a/duchi11a.pdf>
- [4] T. Tieleman and G. Hinton, "Lecture 6.5 - rmsprop: Divide the gradient by a running average of its recent magnitude," *COURSERA: Neural Networks for Machine Learning*, 2012, available at https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>
- [6] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of adam and beyond," *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1904.09237>
- [7] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, "On the variance of the adaptive learning rate and beyond," *International Conference on Learning Representations (ICLR)*, 2020. [Online]. Available: <https://arxiv.org/abs/1908.03265>
- [8] T. Dozat, "Incorporating nesterov momentum into adam," 2016, technical report, Stanford University. [Online]. Available: <https://openreview.net/forum?id=OM0jvwB8jIp57ZJtNEZ>
- [9] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [10] PyTorch Contributors, "Pytorch optimizer documentation," <https://pytorch.org/docs/stable/optim.html>, 2024, accessed: June 2025.
- [11] S. Ruder, "An overview of gradient descent optimization algorithms," <https://arxiv.org/abs/1609.04747>, 2016.
- [12] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of adam and beyond," *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/pdf/1904.09237>
- [13] R. M. Schmidt, F. Schneider, and P. Hennig, "Descending through a crowded valley - benchmarking deep learning optimizers," 2021. [Online]. Available: <https://arxiv.org/abs/2007.01547>

VI. APPENDIX

A. First experimental protocol

Figures 5, 6, 7, 8, and 9 show the training losses for the eight optimizers across different learning rates. Note that, as in Figure 8, the losses for the learning rate of 10^{-1} (Figure 9) were plotted on a logarithmic scale to make the non-diverging curves discernible.

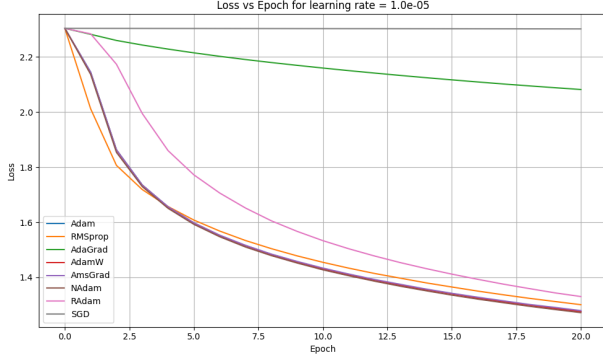


Fig. 5. Loss at initial learning rate 10^{-5}

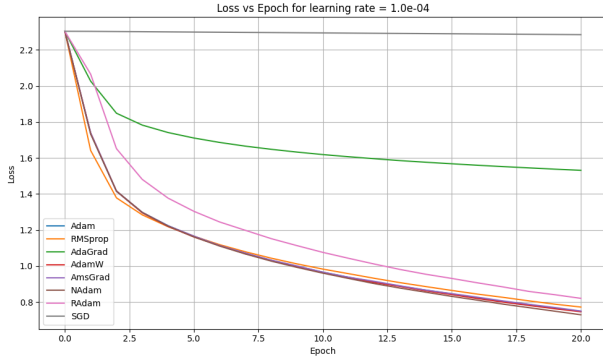


Fig. 6. Loss at initial learning rate 10^{-4}

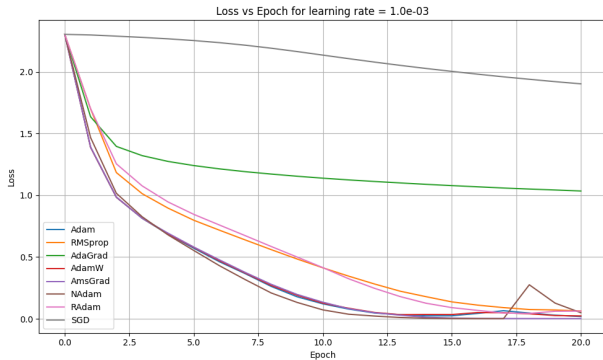


Fig. 7. Loss at initial learning rate 10^{-3}

Figures 10, 11, 12, 13, and 14 show the evolution of accuracy over training epochs for the eight optimizers across

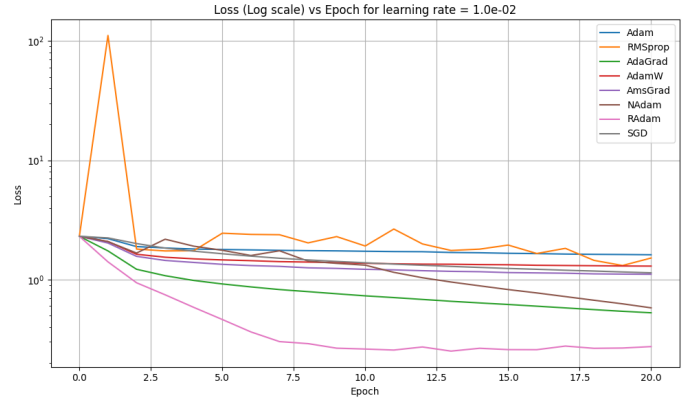


Fig. 8. Loss (Log) at initial learning rate 10^{-2}

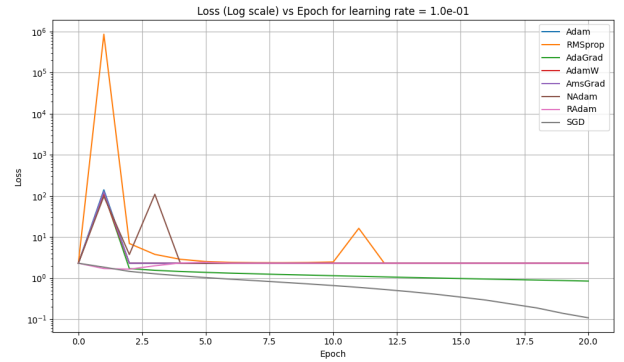


Fig. 9. Loss at initial learning rate 10^{-1}

different learning rates. These plots illustrate well the trade-offs involved in learning rate selection. While the accuracy curves are very smooth with an initial learning rate of 10^{-5} 10, the final accuracy reached after 20 epochs is already outperformed within a single epoch when using a learning rate of 10^{-3} 12.

Within our experimental setup, an initial learning rate of 10^{-2} appears to be the upper bound for stability for most adaptive optimizers 13.

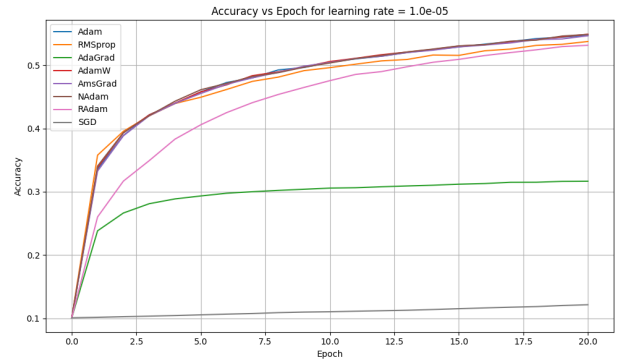


Fig. 10. Accuracy at initial learning rate 10^{-5}

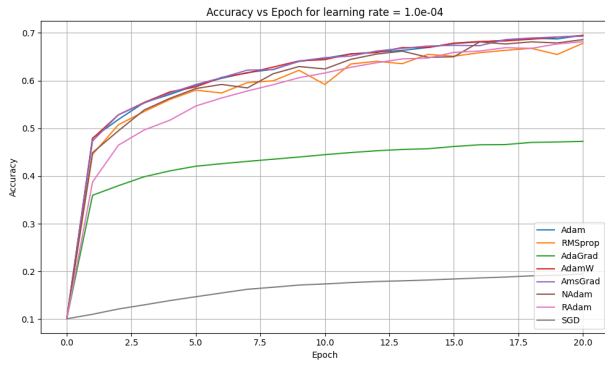


Fig. 11. Accuracy at initial learning rate 10^{-4}

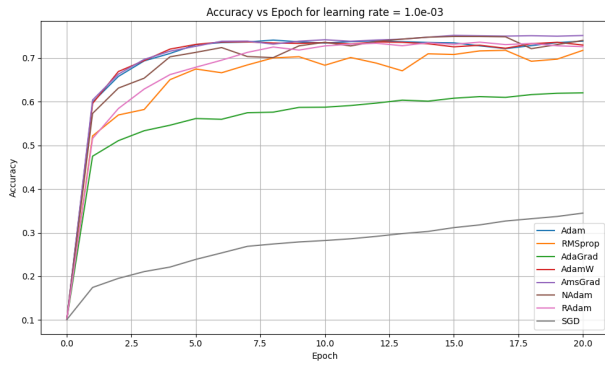


Fig. 12. Accuracy at initial learning rate 10^{-3}

B. Experimental Protocol — Hyperparameter Optimization with Optuna

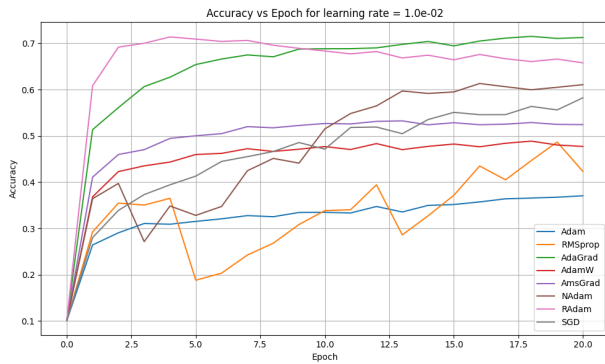


Fig. 13. Accuracy at initial learning rate 10^{-2}

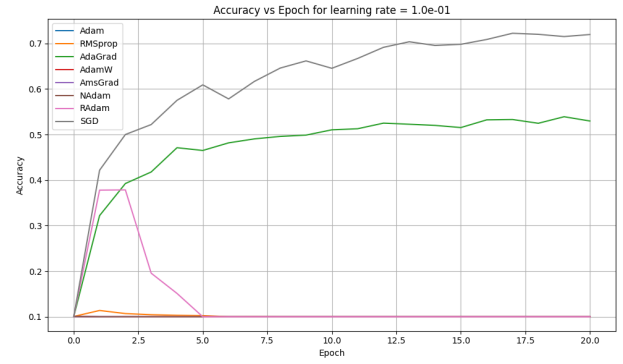


Fig. 14. Accuracy at initial learning rate 10^{-1}

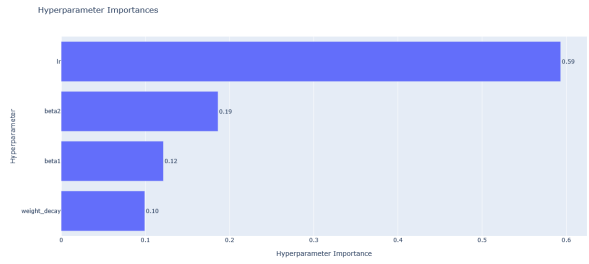


Fig. 15. Importance of RAdam hyperparameters

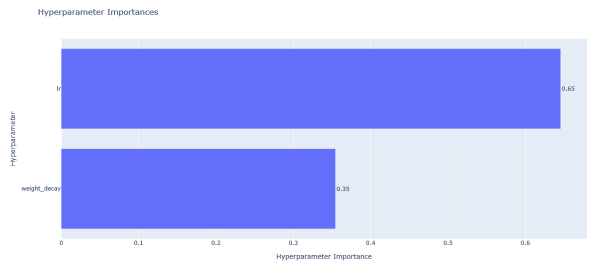


Fig. 16. Importance of AdaGrad hyperparameters